

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Факультет ФТИИ

Дисциплина:
«Frontend-разработка»

Лабораторная № 3

Выполнила:

Кузнецова Таисия
студент группы J3214

(подпись)

Проверила:

...

(отметка о выполнении)

Содержание

1. Цель и постановка задачи
 2. Исходное приложение и связь с лабораторной работой №2
 3. Изменения архитектуры при переходе на Vue.js
 4. Используемые технологии
 5. Структура проекта
 6. Точка входа приложения и главный компонент
 7. Маршрутизация SPA
 8. Компонентная архитектура Vue
 9. Composables и управление состоянием
 10. Работа с mock API и Axios
 11. Реализация основных страниц
 12. Авторизация и регистрация
 13. Сохраненные направления и истории, лайки
 14. Планировщик и интерактивная карта
 15. Темизация и оформление
 16. Адаптивность и доступность
 17. Сравнение лабораторных работ №2 и №3
 18. Проверка и тестирование
 19. Запуск проекта
 20. Результаты работы и заключение
- Приложение А. Таблица файлов и их назначения
- Приложение Б. Ключевые фрагменты кода

1. Цель и постановка задачи

Цель лабораторной работы – перевести ранее разработанное веб-приложение Travelly на Vue.js и организовать его как одностраничное приложение (SPA). При этом основной пользовательский сценарий и уже реализованные возможности должны сохраниться, а структура проекта должна стать компонентной и более удобной для дальнейшего расширения.

В качестве исходной основы использована лабораторная работа №2. В ней Travelly представлял собой набор отдельных HTML-страниц с JavaScript-логикой, mock REST API на JSON Server, авторизацией, поиском направлений, историей путешествий, планировщиком с картой, темизацией и сохранением пользовательских данных.

В лабораторной работе №3 основной акцент сделан именно на Vue-архитектуре. Поэтому задача заключалась не в полном визуальном копировании каждого элемента ЛР2, а в сохранении основных сценариев и переносе их на Vue-компоненты, Vue Router, Composition API, composables и Axios.

Основные требования лабораторной работы

- создать SPA на Vue.js на основе приложения предыдущей лабораторной работы;
- добавить маршрутизацию страниц через Vue Router;
- использовать внешний API, в проекте - mock API на базе JSON Server;
- использовать Axios для HTTP-запросов;
- разделить интерфейс на переиспользуемые Vue-компоненты;
- вынести повторяемую или общую логику в composables;
- сохранить основные функции Travelly: авторизацию, поиск, направления, истории, планировщик, поездки и профиль.

2. Исходное приложение и связь с лабораторной работой №2

Лабораторная работа №2 была основой текущего проекта. В ней Travelly уже имел полноценную структуру сайта: главную страницу, поиск направлений, страницу направления, ленту историй, профиль, мои поездки, регистрацию, вход и планировщик маршрута. Данные хранились в db.json и отдавались через JSON Server. В ЛР2 использовались отдельные HTML-файлы, а логика была распределена между JavaScript-файлами. Например, API было сосредоточено в js/api.js, авторизация - в js/auth.js, переключение темы - в js/theme.js, а общие события и эффекты - в main.js. Навигация выполнялась обычными ссылками между HTML-страницами.

В ЛР3 этот подход заменен на Vue-приложение. Все страницы остаются отдельными представлениями, но фактически загружаются внутри одного корневого приложения через RouterView. Это позволяет переходить между разделами без полноценной перезагрузки HTML-документа.

Элемент ЛР2	Реализация в ЛР2	Реализация в ЛР3
Главная	index.html	HomeView.vue
Поиск	search.html	SearchView.vue
Направление	destination.html	DestinationView.vue + :id
Истории	feed.html	FeedView.vue + PostCard.vue
Профиль	profile.html	ProfileView.vue
Мои поездки	my-trips.html	MyTripsView.vue + TripCard.vue
Планировщик	trip-planner.html	TripPlannerView.vue
Вход / регистрация	login.html / register.html	LoginView.vue / RegisterView.vue
Навигация	HTML-ссылки	Vue Router
API	Fetch API	Axios
Общая логика	main.js, auth.js, theme.js	composables + services
UI-повторное использование	дублирование HTML	Vue-компоненты

3. Изменения архитектуры при переходе на Vue.js

Главное изменение — переход от многостраничной структуры к компонентному SPA. В ЛР2 каждая страница была отдельным HTML-документом. В ЛР3 точкой входа становится main.js, который создает Vue-приложение, подключает роутер и монтирует его в элемент #app.

```
import { createApp } from 'vue'
import App from './App.vue'
import router from './router'
import './assets/main.css'

createApp(App)
  .use(router)
  .mount('#app')
```

Такой подход разделяет приложение на уровни: точка входа отвечает за запуск, App.vue за корневой контейнер, router за навигацию, views за страницы, components за переиспользуемые блоки, composables за общую реактивную логику, services за работу с внешними сервисами.

Главная идея архитектуры

Уровень	Каталог	Назначение
Запуск	src/main.js	Создание и монтирование Vue-приложения
Корневой компонент	src/App.vue	Отображение RouterView
Маршрутизация	src/router	Связь URL и Vue-представлений
Страницы	src/views	Полные пользовательские экраны
Компоненты	src/components	Переиспользуемые визуальные элементы
Общая логика	src/composables	Авторизация, темы, поездки
API	src/services/api.js	Axios и REST-операции
Конфигурация карты	src/services/mapConfig.js	Конфигурация MapTiler
Стили	src/assets/main.css + scoped styles	Общее оформление и локальные стили
Данные	db.json	Mock backend JSON Server

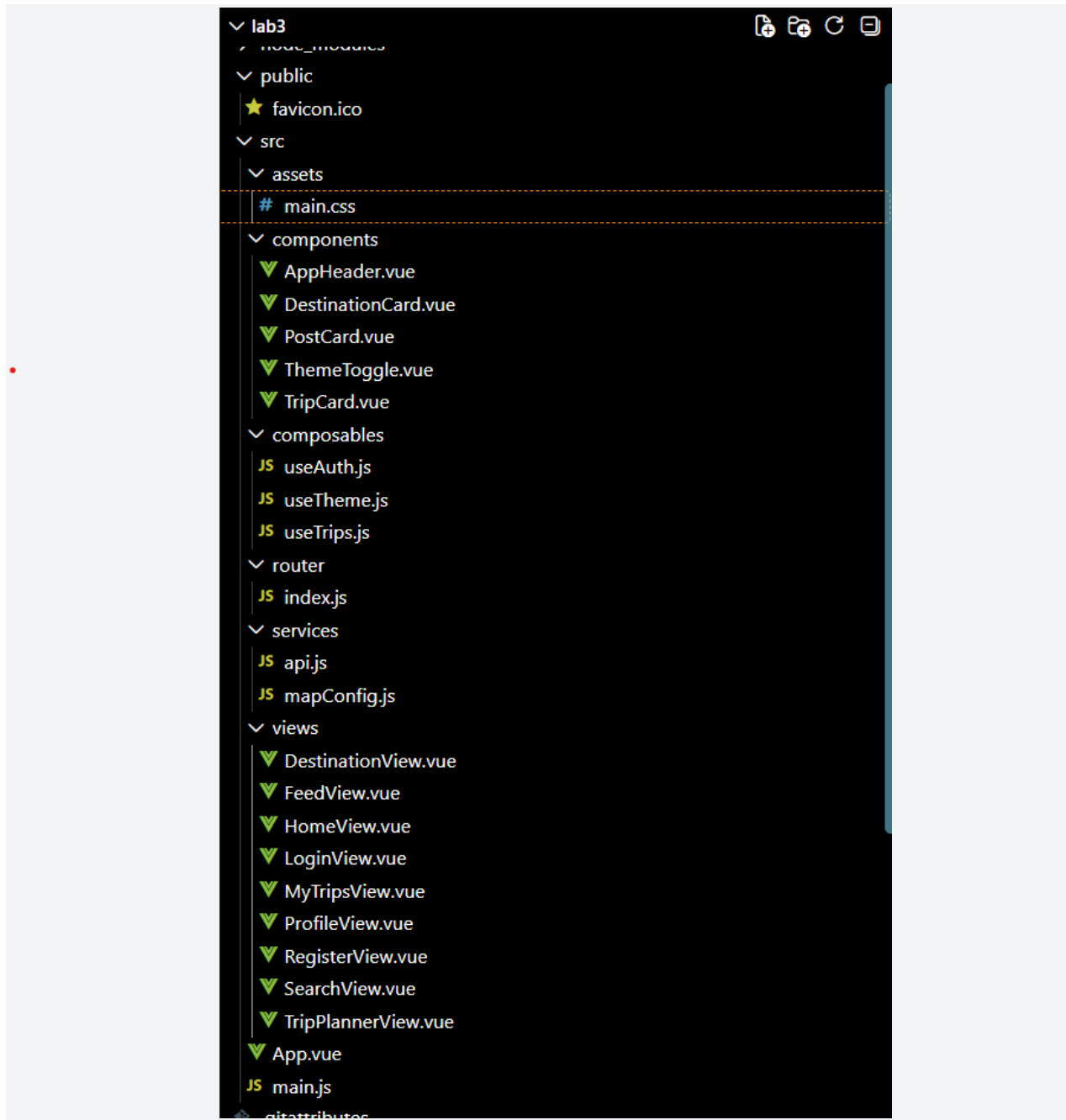
4. Используемые технологии

Технология	Версия / источник	Роль в проекте
Vue	3.5.42	Компонентная разработка интерфейса и реактивное состояние
Vue Router	5.3.1	Маршрутизация SPA
Axios	1.20.0	HTTP-запросы к JSON Server
Vite	8.2.2	Сборка и dev-сервер
@maptiler/sdk	4.1.0	Интерактивная карта
JSON Server	данные db.json	Mock REST API
JavaScript / Composition API	ES modules	Логика компонентов, ref, computed, reactive, lifecycle
CSS3	main.css + scoped styles	Темы, сетки, карточки, адаптивность

В package.json также предусмотрены инструменты ESLint и Oxlint для проверки качества кода. Скрипты проекта включают dev, build, preview и lint. Это

отличается от ЛР2, где package.json в основном использовался для запуска JSON Server.

5. Структура проекта



По структуре видно, что проект организован по назначению, а не только по типу файлов. Это удобно при дальнейшей разработке: изменение логики авторизации не требует поиска кода по нескольким HTML-документам, а общий компонент можно подключить на нужных страницах через `import`.

6. Точка входа приложения и главный компонент

Файл `src/main.js` является точкой запуска Vue-приложения. Здесь импортируются `App.vue`, `router` и общий `main.css`. Через `createApp` создается экземпляр `Vue`, после чего к нему подключается роутер и выполняется `mount` на элемент `#app` из `index.html`.

Компонент `src/App.vue` минимален. Он содержит `RouterView`, в который `Vue Router` подставляет текущее представление в зависимости от URL. Благодаря этому `App.vue` не содержит разметку конкретной страницы и остается корневой оболочкой приложения.

```
<template>
  <div id="app">
    <RouterView />
  </div>
</template>

<script setup>
import { RouterView } from 'vue-router'
</script>
```

7. Маршрутизация SPA

Маршрутизация реализована в `src/router/index.js` с помощью `createRouter` и `createWebHistory`. Каждому URL сопоставляется определенное Vue-представление. Для страницы направления используется динамический параметр `:id`.

```
const routes = [
  { path: '/', name: 'home', component: HomeView },
  { path: '/search', name: 'search', component: SearchView },
  { path: '/destination/:id', name: 'destination', component: DestinationView },
  { path: '/login', name: 'login', component: LoginView },
  { path: '/register', name: 'register', component: RegisterView },
  { path: '/profile', name: 'profile', component: ProfileView },
  { path: '/feed', name: 'feed', component: FeedView },
  { path: '/my-trips', name: 'my-trips', component: MyTripsView },
  { path: '/trip-planner', name: 'trip-planner', component: TripPlannerView }
]
```

Переходы внутри приложения выполнены через `RouterLink`. Это заменяет обычные ссылки вида `index.html` или `search.html` из ЛР2. В `AppHeader.vue` `RouterLink` также используется для выделения активного раздела меню.

8. Компонентная архитектура Vue

В ЛР3 повторяющиеся части интерфейса вынесены в компоненты. Это один из основных результатов перехода на `Vue.js`. В предыдущей версии шапка и часть разметки повторялись на разных HTML-страницах, а теперь `AppHeader.vue` подключается в представлениях как готовый компонент.

Компонент	Где используется	Что делает
AppHeader.vue	почти все страницы	Шапка, навигация, тема, профиль и выход
ThemeToggle.vue	AppHeader	Переключает светлую и темную тему
DestinationCard.vue	HomeView, SearchView, ProfileView	Показывает карточку направления и ведет на /destination/:id
PostCard.vue	HomeView, FeedView, ProfileView	Карточка истории с автором, тегами, лайком и комментариями
TripCard.vue	MyTripsView	Карточка сохраненной поездки, маршрут и удаление

Для передачи данных используются props. Например, DestinationCard.vue принимает объект destination, а PostCard.vue объект post. Для связи дочернего компонента с родительским используются события emit, например событие like-changed при изменении состояния лайка.

Пример props и emit

```
const props = defineProps({
  post: {
    type: Object,
    required: true
  }
})

const emit = defineEmits(['like-changed'])
```

9. Composables и управление состоянием

Для повторно используемой логики в проекте созданы три composable-файла: useAuth.js, useTrips.js и useTheme.js. Они позволяют не дублировать одну и ту же бизнес-логику в представлениях.

useAuth.js

Composable хранит текущего пользователя в реактивной переменной currentUser. При создании composable пользователь восстанавливается из localStorage по ключу travelly-user. Внутри также находятся операции login, register, logout и getUser. После успешного входа или регистрации в localStorage сохраняется сессия с id, name, email и createdAt.

useTrips.js

Composable работает с поездками через tripsApi. Свойство userTrips вычисляется через computed и оставляет только поездки текущего пользователя. Для корректного сравнения id применяется приведение к строке, что позволяет работать и со старыми числовыми id, и с новыми строковыми id.

```
const userTrips = computed(() => {
  if (!currentUser.value) return []

  return trips.value.filter(
    (trip) => String(trip.userId) === String(currentUser.value.id)
  )
})
```

useTheme.js

Composable управляет текущей темой. При старте сначала проверяется сохраненное значение в localStorage, а если его нет — предпочтение системы через prefers-color-scheme. Применение темы выполняется установкой атрибута data-theme на document.documentElement. Это позволяет CSS использовать разные переменные для светлой и темной темы.

```
document.documentElement.setAttribute('data-theme', theme)
localStorage.setItem(STORAGE_KEY, theme)
```

10. Работа с mock API и Axios

В ЛР2 API было реализовано вручную через универсальную функцию request и Fetch API. В ЛР3 этот слой заменен на Axios. В src/services/api.js создается общий axios-экземпляр с baseUrl http://localhost:3000, после чего для каждой сущности определен отдельный объект API.

```
const api = axios.create({
  baseURL: 'http://localhost:3000',
  headers: {
    'Content-Type': 'application/json'
  }
})
```

API-модуль	Основные операции	Коллекция JSON Server
destinationsApi	getAll, getById	destinations
postsApi	getAll, getById, create, update	posts
usersApi	getAll, getById, create, update	users
tripsApi	getAll, create, delete	trips
savedApi	getByUser, create, delete	saved

API-модуль	Основные операции	Коллекция JSON Server
likesApi	getByUser, getByPostAndUser, create, delete	likes

Представления используют этот слой вместо прямых axios-запросов. Например, SearchView.vue вызывает destinationsApi.getAll(), FeedView.vue — postsApi.getAll(), а ProfileView.vue объединяет запросы к trips, saved, likes и posts.

11. Реализация основных страниц

Страница	Основные функции	Vue-механизмы
Главная	hero, популярные направления, карта-предпросмотр, истории, СТА	onMounted, ref, RouterLink, компоненты
Поиск	поиск по названию и фильтры по типу, бюджету, длительности	ref, computed, v-model
Направление	подробная информация, сохранение, идеи мест, СТА в планировщик	route.params.id, computed, API
Истории	загрузка постов и лайки	PostCard, onMounted, emit
Профиль	аккаунт, статистика, сохраненные направления и истории	composables, API, computed
Мои поездки	создание, список и удаление поездок	reactive, useTrips, TripCard
Планировщик	форма поездки, точки маршрута, MapTiler, заметки	reactive, ref, lifecycle, SDK
Вход / регистрация	авторизация и создание аккаунта	v-model, useAuth, RouterLink

Поиск и фильтрация

SearchView.vue загружает все destinations через API и хранит их в ref. Выбранные фильтры также находятся в ref. Результат не записывается отдельным массивом вручную: он вычисляется через computed filteredDestinations. За счет этого интерфейс автоматически обновляется после изменения любого поля.

```
const filteredDestinations = computed(() => {
  const query = searchQuery.value.trim().toLowerCase()

  return destinations.value.filter((destination) => {
    const matchesSearch =
      !query || destination.title.toLowerCase().includes(query)

    const matchesType =
      !selectedType.value || destination.type === selectedType.value
```

```
    return matchesSearch && matchesType
  })
}
```

Страница направления

DestinationView.vue получает id из URL через useRoute(), затем вызывает destinationsApi.getById(id). Состояние загрузки и ошибки хранится в ref. Для сохранения направления используется savedApi: при отсутствии записи создается элемент saved, а при повторном нажатии он удаляется.

Профиль

ProfileView.vue связывает сразу несколько сущностей. Количество поездок считается по userId, сохраненные направления получаются из saved и затем сопоставляются с destinations, а сохраненные истории строятся через likes -> posts. Таким образом, профиль отображает данные, которые действительно находятся в mock API, а не только локальные заглушки.

12. Авторизация и регистрация

Авторизация реализована через useAuth.js и usersApi. При входе загружается список пользователей из JSON Server и выполняется поиск по email и password. Email сравнивается без учета регистра. После успешного входа создается объект сессионных данных и сохраняется в localStorage.

Регистрация сначала проверяет, не существует ли пользователь с таким email. После успешного POST-запроса создается новая сессия. Благодаря общей реактивной переменной currentUser шапка и страницы автоматически видят изменение состояния пользователя.

В отличие от ЛР2, где авторизация была привязана к DOM и отдельному auth.js, в ЛР3 она становится общей логикой Vue-приложения и может использоваться любым представлением без дублирования кода.

13. Сохраненные направления и истории, лайки

Сохранение направления

Для направления используется коллекция saved. В записи хранится userId и destinationId. В DestinationView.vue по текущему пользователю загружаются сохраненные записи и определяется, сохранено ли текущее направление.

Лайки историй

Лайк истории реализован в `PostCard.vue`. Для текущего пользователя загружается список его лайков через `likesApi.getByUser()`, после чего записи фильтруются по `postId`. Это важно для перезагрузки страницы: состояние `liked` восстанавливается из API, а не только из памяти компонента.

При установке лайка создается запись в `likes` и увеличивается поле `likes` у соответствующего поста. При снятии лайка записи пользователя по этой истории удаляются, а счетчик уменьшается. После этого компонент отправляет родителю событие `like-changed`.

```
const existingLikes = await getUserLikes()

if (existingLikes.length > 0) {
  for (const like of existingLikes) {
    await likesApi.delete(like.id)
  }

  liked.value = false
  likesCount.value = Math.max(0, likesCount.value - 1)

  await postsApi.update(props.post.id, {
    likes: likesCount.value
  })

  emit('like-changed', {
    postId: props.post.id,
    liked: false,
    likesCount: likesCount.value
  })
}
```

`ProfileView.vue` использует эти данные для блока «Сохраненные истории». При загрузке профиля запрашиваются `likes` текущего пользователя и все `posts`, после чего выбираются только посты с соответствующим `postId`. Если в профиле снять лайк, `PostCard` отправляет событие, а `handleLikedPostChange` сразу удаляет эту историю из массива `likedPosts`. Получается связанная цепочка: лента -> likes -> профиль -> снятие лайка -> обновление профиля.

14. Планировщик и интерактивная карта

`TripPlannerView.vue` сочетает форму поездки и интерактивную карту. Для формы используется `reactive`, а изменяемые значения вводов связаны с моделью через `v-model`. Список точек маршрута хранится в `ref route`. Новую точку можно добавить кнопкой или клавишей `Enter`, удаление выполняется по индексу.

Карта реализована через `@maptiler/sdk`. Компонент импортирует `Map`, `MapStyle`, `Marker` и `config`, создает карту в DOM-элементе через `ref mapContainer`,

задает стиль STREETS, центр и начальный zoom. Событие click на карте записывается в selectedPoint и добавляет маркер.

```
map.value = new Map({
  container: mapContainer.value,
  style: MapStyle.STREETS,
  center: [37.6173, 55.7558],
  zoom: 4
})

map.value.on('click', handleMapClick)
```

Для работы с заметками используется дополнительный реактивный массив mapNotes. После клика по карте пользователь открывает модальное окно, вводит название и текст, а затем заметка связывается с координатами выбранной точки. При сохранении поездки mapNotes передается вместе с route в createTrip().

Жизненный цикл карты также учитывается: инициализация выполняется в onMounted, а при уничтожении компонента map.remove() вызывается в onBeforeUnmount. Это предотвращает сохранение старого экземпляра карты после перехода на другую страницу.

15. Темизация и оформление

В ЛР2 тема переключалась через отдельный theme.js. В ЛР3 эта логика вынесена в useTheme.js. Состояние темы хранится в ref currentTheme, а выбранное значение сохраняется в localStorage. На корневой HTML-элемент устанавливается data-theme=light или data-theme=dark.

Основные цвета интерфейса заданы через CSS-переменные. Поэтому одинаковые компоненты могут использовать var(--background), var(--card), var(--text), var(--text-secondary), var(--border) и другие значения, не дублируя цвета внутри разных страниц.

Визуальная идея ЛР2 сохранена: сине-белая цветовая схема, крупные карточки направлений с фотографиями, карточки историй, крупная hero-секция, карта, СТА-блоки и общая навигация. При этом реализация интерфейса перенесена в Vue-компоненты и scoped styles.

Пример переключения темы

```
function applyTheme(theme) {
  currentTheme.value = theme
  document.documentElement.setAttribute('data-theme', theme)
  localStorage.setItem(STORAGE_KEY, theme)
}

function toggleTheme() {
```

```

    applyTheme(currentTheme.value === 'dark' ? 'light' : 'dark')
  }

```

16. Адаптивность и доступность

Адаптивность в ЛР3 обеспечивается CSS media queries. Для карточек направлений, историй, планировщика и навигации заданы изменения сеток при уменьшении ширины экрана. Например, трехколоночные сетки переходят в две, а затем в одну колонку.

В компонентном интерфейсе сохраняются семантические элементы form, label, button, main, nav и header. Для интерактивных элементов используются aria-label и title там, где визуального текста недостаточно, например у кнопки темы и кнопки лайка.

При переносе на Vue отдельным плюсом становится более четкая структура: состояние формы и обработчики находятся рядом с шаблоном, поэтому проще отслеживать соответствие label и input, required-полей и доступных действий.

17. Сравнение лабораторных работ №2 и №3

Критерий	ЛР2	ЛР3
Архитектура	Набор HTML-страниц	Одно Vue SPA
Навигация	Переходы между .html	Vue Router и RouterLink
UI	HTML + Bootstrap + CSS	Vue SFC + CSS
Повторяемые блоки	Повторение разметки	AppHeader, карточки, ThemeToggle
Состояние	DOM + localStorage	ref, reactive, computed, localStorage
Общая логика	JS-файлы	Composables
API	Fetch API	Axios
Страница направления	destination.html	DestinationView + динамический :id
Истории	HTML + main.js	FeedView + PostCard
Лайки	клиентская логика DOM	likes в API + реактивное состояние
Профиль	DOM + JS	композиция API-запросов и компоненты
Планировщик	JS + HTML	Vue state + MapTiler SDK
Сборка	фактически статические страницы	Vite
Проверка кода	обычные JS-файлы	ESLint / Oxlint предусмотрены в npm scripts

Что именно стало лучше с точки зрения структуры

- одно изменение в AppHeader.vue влияет на весь сайт, а не на набор HTML-файлов;
- одна и та же карточка направления используется на главной, в поиске и профиле;
- авторизация, тема и работа с поездками вынесены в повторно используемые composables;
- HTTP-запросы централизованы в services/api.js;
- страница направления стала параметризованной через /destination/:id;
- реактивность Vue автоматически обновляет интерфейс при изменении состояния.

Что изменилось визуально

В ЛР3 визуальная концепция ЛР2 сохранена, но часть деталей оформлена по-другому из-за перехода на компонентную структуру. Это не меняет пользовательский сценарий: основные разделы и действия остались теми же. Дополнительно были доработаны состояния темной темы, отображение сохраненных историй и страница подробного направления.

18. Проверка и тестирование

В ходе разработки основные пользовательские сценарии проверялись в браузере после запуска Vite и JSON Server. Отдельно проверялись светлая и темная темы, навигация, поиск, сохранение направлений, создание поездок, работа карты и лайков.

Сценарий	Что проверяется	Ожидаемый результат
Запуск	Vite + JSON Server	Приложение открывается, API доступен на :3000
Вход	email и пароль	Текущий пользователь появляется в шапке
Регистрация	новый email	Пользователь создается через API
Поиск	строка и фильтры	Список направлений меняется реактивно
Направление	/destination/:id	Данные загружаются по id
Сохранение	кнопка Сохранить	Запись появляется в saved и профиле

Сценарий	Что проверяется	Ожидаемый результат
Лайк	кнопка ♥	Запись в likes сохраняется, счетчик меняется
Обновление после лайка	перезагрузка страницы	Лайк остается активным
Снятие лайка	повторный клик	Запись likes удаляется и история исчезает из профиля
Тема	переключатель	Меняются CSS-переменные, тема сохраняется
Карта	клик по карте	Появляется marker и можно создать заметку
Поездка	сохранение формы	Поездка появляется в Мои поездки
Удаление поездки	кнопка ×	Запись удаляется из API и списка

Важное практическое отличие от ЛР2 - в ЛР3 состояние интерфейса тесно связано с состоянием API. Например, лайк не считается установленным только из-за CSS-класса: при загрузке PostCard снова обращается к likes и восстанавливает состояние. Это делает поведение устойчивым к перезагрузке страницы.

19. Запуск проекта

Для запуска проекта необходимо открыть терминал в корневой папке lab3. После установки зависимостей запускаются два процесса: JSON Server для API и Vite для клиентской части.

```
npm install

# терминал 1
npx json-server db.json --port 3000

# терминал 2
npm run dev
```

После запуска Vite в терминале выводится локальный адрес приложения. JSON Server работает на `http://localhost:3000` и предоставляет коллекции из `db.json`. Для production-сборки предусмотрена команда `npm run build`, а для локального просмотра готовой сборки — `npm run preview`.

API-ключ MapTiler хранится в `src/services/mapConfig.js`. В отчет ключ намеренно не включается.

20. Результаты работы и заключение

В результате приложение Travelly перенесено с многостраничной структуры ЛР2 на Vue 3 и организовано как SPA. Сохранены основные сценарии предыдущей версии: регистрация и вход, поиск направлений с фильтрацией, страница направления, лента историй, профиль, сохраненные направления, мои поездки и планировщик с интерактивной картой. Главным результатом является не только изменение синтаксиса HTML на Vue-шаблоны. Архитектура стала компонентной: AppHeader, DestinationCard, PostCard, TripCard и ThemeToggle можно повторно использовать на разных страницах. Повторяемая логика вынесена в useAuth, useTrips и useTheme, а работа с backend сосредоточена в Axios-слое services/api.js.

По сравнению с ЛР2 приложение стало более удобным для дальнейшего расширения. Для добавления новой страницы достаточно создать view и маршрут, для нового повторяемого блока компонент, а общую бизнес-логику можно вынести в composable или API-модуль. Это соответствует цели перехода к современному SPA-подходу на Vue.js.

Отдельно была доработана социальная часть. Лайк истории теперь хранится в mock API в коллекции likes, счетчик обновляется в posts, а профиль собирает сохраненные истории по лайкам текущего пользователя. Поэтому взаимодействие пользователя стало связанным между страницами и сохраняется после обновления браузера.

Итоговая версия соответствует задаче лабораторной работы №3: используется Vue.js, Vue Router, Axios, компонентная структура, composables и внешний mock API. При этом проект сохраняет назначение Travelly и основу пользовательских сценариев, созданных в лабораторной работе №2.

Приложение А. Таблица файлов и их назначения

Файл	Назначение	Ключевые элементы
src/main.js	Запуск Vue	createApp, use(router), mount
src/App.vue	Корень приложения	RouterView
src/router/index.js	Маршрутизация	createRouter, createWebHistory, routes
src/services/api.js	API	Axios, REST-операции
src/services/mapConfig.js	Карта	MapTiler API key
src/composables/useAuth.js	Сессия	ref, computed, localStorage, usersApi
src/composables/useTrips.js	Поездки	computed, tripsApi, CRUD
src/composables/useTheme.js	Темизация	ref, localStorage, data-theme
src/components/AppHeader.vue	Шапка	RouterLink, ThemeToggle, useAuth
src/components/DestinationCard.vue	Карточка направления	props, RouterLink
src/components/PostCard.vue	История	props, refs, likesApi, postsApi, emit
src/components/TripCard.vue	Поездка	props, computed, emit
src/components/ThemeToggle.vue	Тема	useTheme
src/views/HomeView.vue	Главная	API, cards, RouterLink
src/views/SearchView.vue	Поиск	v-model, computed, API
src/views/DestinationView.vue	Направление	route params, savedApi, content blocks
src/views/FeedView.vue	Истории	postsApi, PostCard
src/views/ProfileView.vue	Профиль	savedApi, likesApi, postsApi, tripsApi
src/views/MyTripsView.vue	Мои поездки	useTrips, TripCard, форма создания
src/views/TripPlannerView.vue	Планировщик	MapTiler, reactive, route, notes
src/views/LoginView.vue	Вход	useAuth, RouterLink
src/views/RegisterView.vue	Регистрация	useAuth, form validation
src/assets/main.css	Общие стили	цветовые переменные, тема, layout, responsive

Приложение Б. Ключевые фрагменты кода

Б.1. Подключение роутера

```
createApp(App)
  .use(router)
  .mount('#app')
```

Б.2. Получение данных через Axios

```
export const destinationsApi = {
  async getAll() {
    const response = await api.get('/destinations')
    return response.data
  },

  async getById(id) {
    const response = await api.get(`/destinations/${id}`)
    return response.data
  }
}
```

Б.3. Реактивная фильтрация

```
const searchQuery = ref('')
const selectedType = ref('')

const filteredDestinations = computed(() => {
  const query = searchQuery.value.trim().toLowerCase()
  return destinations.value.filter((destination) => {
    const matchesSearch =
      !query || destination.title.toLowerCase().includes(query)
    const matchesType =
      !selectedType.value || destination.type === selectedType.value
    return matchesSearch && matchesType
  })
})
```

Б.4. Передача данных в компонент

```
<DestinationCard
  v-for="destination in destinations"
  :key="destination.id"
  :destination="destination"
/>
```